



Disciplina: Análise E Complexidade De Algoritmos

Grafos: Representação, BFS, DFS e Dijkstra

Igor Jacomossi Riberto - BI300094X

Jorge Luiz Marques Bom Junior - BI3012018

Júlio Cesar de Oliveira Geraldo - BI300399X

Lucas Alves De Souza – BI3003698

Vitor da Silva Campos - BI3005577

Birigui

2026

Resumo

Este relatório apresenta o estudo e a implementação de estruturas e algoritmos fundamentais sobre grafos, desenvolvido no contexto da disciplina de Algoritmos e Estruturas de Dados. São abordados os conceitos formais de grafos, as representações por matriz e lista de adjacência, os algoritmos de busca em largura (BFS) e profundidade (DFS), e o algoritmo de Dijkstra para caminhos mínimos em grafos ponderados. As implementações foram realizadas em C# com .net8.0, validadas com casos de teste, e aplicadas a um grafo representando um mapa de cidades com 10 vértices. A análise de complexidade de cada algoritmo é apresentada de forma formal. O relatório também inclui a resolução do Desafio de Programação Competitiva proposto, com comparação entre as abordagens com heap e vetor simples para o algoritmo de Dijkstra.

Palavras-chave: Grafos. BFS. DFS. Dijkstra. Bellman-Ford. Caminho Mínimo. Complexidade de Algoritmos.

Sumário

1. INTRODUÇÃO	3
2. REFERENCIAL TEÓRICO	4
2.1 Definição Formal de Grafo.....	5
2.2 Representações de Grafos	5
2.2.1 Matriz de Adjacência	5
2.2.2 Lista de Adjacência	6
2.3 Algoritmo BFS (Busca em Largura)	6
2.4 Algoritmo DFS (Busca em Profundidade)	6
2.5 Algoritmo de Dijkstra.....	7
2.6 Algoritmo de Bellman-Ford.....	7
3.1 Visão Geral	9
3.2 Representações de Grafo.....	10
3.3 BFS e DFS	11
3.4 Algoritmo de Dijkstra.....	13
3.5 Mapa de Cidades.....	14
4. ANÁLISE DE COMPLEXIDADE	15
4.1 BFS e DFS	16
4.2 Dijkstra com Heap Mínimo	16
4.3 Comparação: Heap vs. Vetor	17
4.4 Bellman-Ford	18
5. DESAFIO DE PROGRAMAÇÃO COMPETITIVA	18
5.1 Caminho Mínimo com Obstáculos.....	19
5.2 Abordagem de Solução	19
5.3 Análise da Complexidade.....	19
5.4 Casos de Teste	20
5.5 Outro Problema que se Reduz a Caminho Mínimo	21
6.1 Por que o Dijkstra Falha com Pesos Negativos	23
6.2 Como o Bellman-Ford Resolve.....	24
7. RESULTADOS E DISCUSSÃO	24
7.1 Verificação de Conectividade, Ciclos e Componentes Conexas (BFS/DFS)	25

7.2 Caminhos Mínimos no Mapa de Cidades	25
7.3 Comparação Empírica Heap vs. Vetor	27
8. CONCLUSÃO	27
REFERÊNCIAS.....	28

1. INTRODUÇÃO

Nesse trabalho vamos apresentar a definição e algumas estruturas de grafos, que são utilizadas para modelar relações entre objetos. Muitos problemas podem ser resolvidos através de grafos como por exemplo roteamento de redes, navegação de GPS, análise de redes sociais, entre outras. Alguns tópicos abordados são:

- Definição e representação computacional de grafos;
- Algoritmos de Busca em Largura (BFS) e em Profundidade (DFS);
- O algoritmo Dijkstra para cálculos de caminhos mínimos;
- O algoritmo Bellman-Ford e quando é utilizado;
- Resolução de um desafio de programação competitiva.

O restante deste relatório está organizado da seguinte forma: a Seção 2 apresenta o referencial teórico; a Seção 3 descreve as implementações realizadas; a Seção 4 apresenta os resultados e análises; a Seção 5 resolve o desafio de programação competitiva; a Seção 6 discute Dijkstra versus Bellman-Ford; e a Seção 7 traz alguns resultados dos algoritmos; e no 8 as conclusões.

2. REFERENCIAL TEÓRICO

2.1 Definição Formal de Grafo

Um grafo G é definido formalmente como um par $G = (V, A)$, onde V é um conjunto de vértices e A é um conjunto de arestas que representam relações entre pares de vértices. Quando as arestas possuem direção, o grafo é chamado de direcionado. Caso contrário, é não-direcionado (CORMEN, 2017).

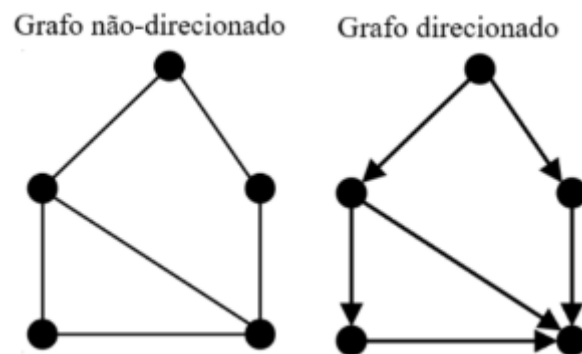


Figura 1: Grafos não direcionados e direcionados.

Em grafos ponderados, cada aresta possui um peso, geralmente representando custo, distância ou capacidade. O grau de um vértice em um grafo não-direcionado é o número de arestas incidentes a ele.

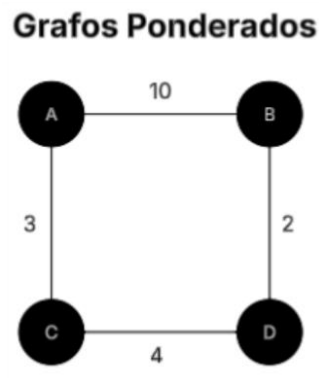


Figura 2: Grafos ponderados.

2.2 Representações de Grafos

As duas principais representações de grafos são a matriz de adjacência e a lista de adjacência, cada uma com vantagens distintas dependendo da aplicação.

2.2.1 Matriz de Adjacência

A matriz de adjacência de um grafo $G = (V, A)$ é uma matriz M de tamanho $|V| \times |V|$, onde $M[i][j] = 1$, se existe aresta de i para j , e 0 caso contrário. O espaço requerido é $\Theta(V^2)$. Consulta de existência de aresta: $O(1)$. Enumeração de todos os vizinhos: $O(V)$. Mais adequada para grafos densos onde $|A| \approx |V|^2$.

2.2.2 Lista de Adjacência

Na lista de adjacência, cada vértice possui uma lista encadeada (ou vetor) com todos os seus vizinhos. O espaço total é $\Theta(V + A)$. Consulta de existência de aresta: $O(\text{grau}(v))$. Enumeração dos vizinhos: $O(\text{grau}(v))$. Mais adequada para grafos esparsos onde $|A| \ll |V|^2$.

Critério	Matriz / Lista
Espaço	$O(V^2)$ / $O(V + A)$
Verificar aresta (u, v)	$O(1)$ / $O(\text{grau}(u))$
Listar vizinhos de u	$O(V)$ / $O(\text{grau}(u))$
Mais indicada para	Grafo denso / Grafo esperso

2.3 Algoritmo BFS (Busca em Largura)

O algoritmo BFS explora o grafo a partir de um vértice, visitando todos os vértices alcançáveis em ordem crescente de distância. Utiliza uma fila como estrutura auxiliar e mantém para cada vértice. A complexidade do BFS é $O(V + A)$, cada vértice é enfileirado e desenfileirado no uma vez ($O(V)$), e cada lista de adjacência é percorrida completamente ($O(A)$).

Pseudocódigo BFS(G, s)

BFS(G, s):

para todo $v \in V[G] - \{s\}$:

$\text{cor}[v] \leftarrow \text{BRANCO}$; $d[v] \leftarrow \infty$; $\pi[v] \leftarrow \text{NIL}$

$\text{cor}[s] \leftarrow \text{CINZA}$; $d[s] \leftarrow 0$; $\pi[s] \leftarrow \text{NIL}$

$Q \leftarrow \{\}$

ENQUEUE(Q, s)

enquanto $Q \neq \{\}$:

$u \leftarrow \text{DEQUEUE}(Q)$

 para todo $v \in \text{Adj}[u]$:

 se $\text{cor}[v] = \text{BRANCO}$:

$\text{cor}[v] \leftarrow \text{CINZA}$; $d[v] \leftarrow d[u] + 1$; $\pi[v] \leftarrow u$

 ENQUEUE(Q, v)

$\text{cor}[u] \leftarrow \text{PRETO}$

2.4 Algoritmo DFS (Busca em Profundidade)

O algoritmo DFS explora o grafo indo o mais fundo possível antes de retroceder. Para cada vértice são registrados os instantes de descoberta e finalização e permitem classificar as arestas do grafo. A estrutura resultante é a floresta DFS. A complexidade total do DFS é igualmente $O(V + A)$.

Pseudocódigo DFS(G)

DFS(G):

para todo $u \in V[G]$: $cor[u] \leftarrow \text{BRANCO}$; $\pi[u] \leftarrow \text{NIL}$

tempo $\leftarrow 0$

para todo $u \in V[G]$:

se $cor[u] = \text{BRANCO}$: DFS-VISIT(G, u)

DFS-VISIT(G, u):

$cor[u] \leftarrow \text{CINZA}$; tempo $\leftarrow \text{tempo} + 1$; $d[u] \leftarrow \text{tempo}$

para todo $v \in \text{Adj}[u]$:

se $cor[v] = \text{BRANCO}$: $\pi[v] \leftarrow u$; DFS-VISIT(G, v)

$cor[u] \leftarrow \text{PRETO}$; tempo $\leftarrow \text{tempo} + 1$; $f[u] \leftarrow \text{tempo}$

2.5 Algoritmo de Dijkstra

O algoritmo de Dijkstra resolve o problema de caminhos mínimos em grafos com pesos não-negativos. Mantém os vértices com distâncias já definitivas e uma fila de prioridade com os demais vértices, ordenados pela distância. A técnica de relaxamento atualiza sempre que este valor for menor que a estimativa atual. A complexidade é $O((V + A) \log V)$, pois cada vértice é extraído uma vez e cada aresta pode causar uma atualização no heap.

Pseudocódigo DIJKSTRA(G, w, s)

DIJKSTRA(G, w, s):

INITIALIZE-SINGLE-SOURCE(G, s)

$S \leftarrow \{\}$

$Q \leftarrow V[G]$ // fila de prioridade (heap mínimo) por $d[v]$

enquanto $Q \neq \{\}$:

$u \leftarrow \text{EXTRACT-MIN}(Q)$

$S \leftarrow S \cup \{u\}$

para todo $v \in \text{Adj}[u]$:

RELAX(u, v, w)

RELAX(u, v, w):

se $d[v] > d[u] + w(u, v)$:

$d[v] \leftarrow d[u] + w(u, v)$

$\pi[v] \leftarrow u$

2.6 Algoritmo de Bellman-Ford

Ao contrário do Dijkstra, o Bellman-Ford suporta arestas com pesos negativos e detecta ciclos de peso negativo. Realiza $|V| - 1$ iterações de relaxamento sobre todas as arestas, com complexidade $O(V \times A)$. Em grafos com pesos não-negativos, Dijkstra é preferível por sua eficiência superior.

3. IMPLEMENTAÇÃO

3.1 Visão Geral

O programa foi desenvolvido em C# (.NET 8) e organizado em pastas conforme a funcionalidade. Cada algoritmo possui uma classe de execução e uma classe de análise que exibe os resultados formatados no console. O menu principal permite escolher a operação desejada. Durante o desenvolvimento, utilizamos o ChatGPT como ferramenta de apoio para estruturar as análises e gerar trechos de código, que foram posteriormente adaptados e integrados pelo grupo. O código-fonte completo está disponível no repositório GitHub do grupo: [algoritmos-grafos-grupo5](#)

A estrutura do repositório está organizada da seguinte forma:

Estrutura do Repositório	
ProjetoGrafos/	
├─	Program.cs
├─	Comparacao/
├─	DesempenhoDijkstra.cs
├─	Bfs/
├─	BFS.cs
├─	AnalizadorBFS.cs
├─	Desafio/
├─	Dijkstra/
├─	Dijkstra.cs
├─	DijkstraVector.cs
├─	AnalizadorDijkstra.cs
├─	Dfs/
├─	DFS.cs
├─	AnalizadorDFS.cs
├─	Grafos/
├─	GrafoMatrizAdjacencia.cs
├─	AnalizadorMatriz.cs
├─	GrafoListaAdjacencia.cs
├─	AnalizadorLista.cs

└─ MapaCidades/
└─ MapaCidades.cs
README.md

3.2 Representações de Grafo

Foram implementadas duas representações de grafos: matriz de adjacência e lista de adjacência. A primeira é mais adequada para grafos densos, pois permite verificar a existência de uma aresta em $O(1)$ e ocupa $O(V^2)$ de memória. A segunda é para grafos esparsos, pois ocupa $O(V + A)$ e permite percorrer os vizinhos de um vértice em tempo proporcional ao seu grau.

A Figura 3 mostra a análise de um grafo denso (arquivo Grafo/1_grafo_denso.txt) utilizando a matriz de adjacência demonstrado na opção 1 no código do GitHub. É possível observar a densidade, o espaço ocupado, os graus dos vértices e a própria matriz. Grafo representado por MATRIZ DE ADJACÊNCIA.

Código:

Classe do grafo: Grafos/GrafoMatrizAdjacencia.cs

Análise: Grafos/AnalizadorMatriz.cs

```
=====
ANÁLISE DO GRAFO (MATRIZ)
=====

[1] CARACTERÍSTICAS BÁSICAS
Vértices (|V|) = 10
Direcionado = Não
Arestas (|A|) = 45
Densidade = 1,0000 (100,00%)
Classificação = DENSO (recomendado: MATRIZ)

[2] ESPAÇO OCUPADO (memória)
Matriz: 0,39 KB (O(V²))
Lista : 0,59 KB (O(V + A))
Economia (matriz vs. lista): 0,20 KB

[3] GRAUS DOS VÉRTICES
Vértice | Grau
-----|-----
0 | 9
1 | 9
2 | 9
3 | 9
4 | 9
5 | 9
6 | 9
7 | 9
8 | 9
9 | 9

[4] MATRIZ DE ADJACÊNCIA
    0  1  2  3  4  5  6  7  8  9
0 | .  5  3  7  2  8  1  6  4  9
1 | 5  .  6  4  9  1  7  3  8  2
2 | 3  6  .  5  8  3  2  6  9  1
3 | 7  4  5  .  1  9  4  8  3  7
4 | 2  9  8  1  .  7  2  5  9  1
5 | 8  1  3  9  7  .  8  3  6  4
6 | 1  7  2  4  2  8  .  1  5  8
7 | 6  3  6  8  5  3  1  .  9  2
8 | 4  8  9  3  9  6  5  9  .  3
9 | 9  2  1  7  1  4  8  2  3  .

[5] EXEMPLO DE CONSULTA
Vizinhos do vértice 0: 1, 2, 3, 4, 5, 6, 7, 8, 9
Grau do vértice 0: 9
Aresta 0-1: existe (peso 5)

[6] RECOMENDAÇÃO
✅ Use MATRIZ DE ADJACÊNCIA: grafos densos, consultas O(1).
```

Figura 3: Grafo representado por Matriz de Adjacência.

A lista de adjacência é carregada do mesmo formato de arquivo (Grafos/2_grafo_esparso.txt (grafo esparso com 10 vértices)). A análise apresenta as mesmas

informações da matriz, mas com a visualização das listas de vizinhos na opção 2. A Figura 4 ilustra a saída para o grafo esparsos. Grafo representado por LISTA DE ADJACÊNCIA.

Código:

Classe do grafo: Grafos/GrafoListaAdjacencia.cs

Análise: Grafos/AnalizadorLista.cs

```
=====
ANÁLISE DO GRAFO (LISTA)
=====

[1] CARACTERÍSTICAS BÁSICAS
Vértices (|V|) = 10
Direcionado = Não
Arestas (|A|) = 13
Densidade = 0,2889 (28,89%)
Classificação = MEDIANAMENTE DENSO

[2] ESPAÇO OCUPADO (memória)
Lista : 0,44 KB (O(V + A))
Matriz: 0,39 KB (O(V²))
Economia (matriz vs. lista): 0,05 KB

[3] GRAUS DOS VÉRTICES
Vértice | Grau
-----|-----
0 | 3
1 | 3
2 | 3
3 | 3
4 | 2
5 | 2
6 | 3
7 | 3
8 | 2
9 | 2

[4] LISTA DE ADJACÊNCIA
0: ->(1,5) ->(2,3) ->(3,7)
1: ->(0,5) ->(4,2) ->(5,8)
2: ->(0,3) ->(6,1) ->(7,9)
3: ->(0,7) ->(8,4) ->(9,6)
4: ->(1,2) ->(6,5)
5: ->(1,8) ->(7,3)
6: ->(2,1) ->(4,5) ->(8,7)
7: ->(2,9) ->(5,3) ->(9,2)
8: ->(3,4) ->(6,7)
9: ->(3,6) ->(7,2)

[5] EXEMPLO DE CONSULTA
Vizinhos do vértice 0: 1(5), 2(3), 3(7)
Grau do vértice 0: 3
Aresta 0+1: existe (peso 5)

[6] RECOMENDAÇÃO
✅ Use LISTA DE ADJACÊNCIA: grafos esparsos, economiza memória.
```

Figura 4: Grafo representado por Lista de Adjacência.

3.3 BFS e DFS

A Busca em Largura (BFS) e a Busca em Profundidade (DFS) foram implementadas conforme o pseudocódigo clássico. A Figura 5 apresenta a execução da opção 3 da Busca em Largura a partir do vértice 0 (Grafos/10_grafo_para_bfs_dfs.txt (grafo com 6 vértices, próprio para testes de busca)). São exibidas as distâncias, a árvore BFS e os níveis (camadas) da busca. Representação do algoritmo BFS (Busca em Largura).

Código:

Execução: Bfs/BFS.cs

Análise: Bfs/AnalizadorBFS.cs

```

=====
BUSCA EM LARGURA (BFS)
=====

Origem: vértice 0

Vértice | Distância | Predecessor | Nível
-----|-----|-----|-----
0       | 0         | -           | 1
1       | 1         | 0           | 2
2       | 1         | 0           | 2
3       | 2         | 1           | 3
4       | 2         | 2           | 3
5       | 3         | 3           | 4

ÁRVORE BFS (predecessores):
Raiz: 0
0 → 1 (distância: 1)
0 → 2 (distância: 1)
1 → 3 (distância: 2)
2 → 4 (distância: 2)
3 → 5 (distância: 3)

NÍVEIS DA BFS:
Nível 0: 0
Nível 1: 1, 2
Nível 2: 3, 4
Nível 3: 5

Todos os vértices são alcançáveis a partir da origem.
=====

```

Figura 5: Algoritmo BFS.

O DFS é executado recursivamente e registra tempos de descoberta e finalização. As arestas são classificadas em árvore, retorno, avanço e cruzamento. Foi utilizado o mesmo grafo do BFS (Grafos/10_grafo_para_bfs_dfs.txt). A Figura 6 apresenta a saída com a classificação completa e indica se há ciclos na opção 4. Representação do algoritmo DFS (Busca em Profundidade).

Código:

Execução: Dfs/DFS.cs

Análise: Dfs/AnalizadorDFS.cs

```

=====
BUSCA EM PROFUNDIDADE (DFS)
=====

Vértice | Descoberta (d) | Finalização (f) | Predecessor
-----|-----|-----|-----
0       | 1             | 12              | -
1       | 2             | 11              | 0
2       | 6             | 7               | 4
3       | 3             | 10              | 1
4       | 5             | 8               | 5
5       | 4             | 9               | 3

ÁRVORE DFS (predecessores):
Raiz: 0
0 → 1
4 → 2
1 → 3
5 → 4
3 → 5

CLASSIFICAÇÃO DAS ARESTAS:
Retorno:
1 → 0
3 → 1
5 → 3
2 → 0
2 → 4
4 → 5
Árvore:
4 → 2
5 → 4
3 → 5
1 → 3
0 → 1
Avanço:
0 → 2

CICLOS: SIM (detectado)
=====

```

Figura 6: Algoritmo DFS.

O BFS é utilizado para verificar a conectividade do grafo e calcular distâncias mínimas em número de arestas, enquanto o DFS permite detectar ciclos e identificar componentes fortemente conexas. Ambos os algoritmos têm complexidade $O(V + A)$.

3.4 Algoritmo de Dijkstra

O algoritmo de Dijkstra foi implementado para encontrar os caminhos mínimos a partir de um vértice origem em grafos com pesos não negativos. A implementação utiliza uma fila de prioridade (heap mínimo) para extrair sempre o vértice com menor distância provisória, conforme a técnica de relaxamento apresentada na aula. A complexidade do algoritmo é $O((V + A) \log V)$.

Código:

Execução: Dijkstra/ AlgoritmoDijkstra.cs

Análise: Dijkstra/AnalizadorDijkstra.cs

A análise exibe:

- Tabela com distâncias mínimas e predecessores.
- Árvore de caminhos mínimos (predecessores).
- Caminho mínimo detalhado da origem até cada vértice alcançável.
- Vértices inalcançáveis (se houver).

A Figura 7 mostra a saída completa da execução do Dijkstra na opção 5.

```
=====
DIJKSTRA - CAMINHO MÍNIMO
=====

Origem: vértice 0

Vértice | Distância | Predecessor
-----|-----|-----
0       | 0         | -
1       | 10        | 0
2       | 15        | 0
3       | 22        | 1
4       | 23        | 2
5       | 33        | 2
6       | 28        | 4
7       | 35        | 0
8       | 55        | 5
9       | 51        | 7

ÁRVORE DE CAMINHOS MÍNIMOS (predecessores):
Raiz: 0
0 → 1 (distância: 10)
0 → 2 (distância: 15)
1 → 3 (distância: 22)
2 → 4 (distância: 23)
2 → 5 (distância: 33)
4 → 6 (distância: 28)
0 → 7 (distância: 35)
5 → 8 (distância: 55)
7 → 9 (distância: 51)

CAMINHOS MÍNIMOS A PARTIR DA ORIGEM:
0 → 1: 0 → 1 (distância: 10)
0 → 2: 0 → 2 (distância: 15)
0 → 3: 0 → 1 → 3 (distância: 22)
0 → 4: 0 → 2 → 4 (distância: 23)
0 → 5: 0 → 2 → 5 (distância: 33)
0 → 6: 0 → 2 → 4 → 6 (distância: 28)
0 → 7: 0 → 7 (distância: 35)
0 → 8: 0 → 2 → 5 → 8 (distância: 55)
0 → 9: 0 → 7 → 9 (distância: 51)

Todos os vértices são alcançáveis a partir da origem.
=====
```

Figura 7: Algoritmo Dijkstra com Heap Mínimo.

A versão com vetor simples (não mostrada aqui) substitui a fila de prioridade por uma varredura linear para encontrar o vértice de menor distância, resultando em complexidade $O(V^2 + A)$. A comparação entre as duas versões é discutida na Seção 4.

3.5 Mapa de Cidades

Para demonstrar o algoritmo de Dijkstra em um contexto prático, modelamos um grafo ponderado representando um mapa rodoviário com 10 cidades brasileiras e 15 rodovias. Os pesos representam as distâncias aproximadas em quilômetros entre as cidades, obtidas a partir de dados reais de rotas rodoviárias.

Código:

MapaCidades/MapaCidades.cs

Execução (Dijkstra): Dijkstra/AlgoritmoDijkstra.cs

Análise: Dijkstra/AnalizadorDijkstra.cs

Tabela de grafo ponderado representando um mapa rodoviário		
Vértice	Cidade Representada	Estado
0	São Paulo	SP
1	Rio de Janeiro	RJ
2	Belo Horizonte	MG
3	Brasília	DF
4	Goiânia	GO
5	Curitiba	PR
6	Porto Alegre	RS
7	Salvador	BA
8	Aracaju	SE
9	Fortaleza	CE

Tabela de visualização do grafo (arestas com pesos reais).			
Origem	Destino	Distância (Km)	Rodovia aproximada
0 (São Paulo)	1 (Rio de Janeiro)	400	BR-116 / Dutra
0 (São Paulo)	2 (Belo Horizonte)	580	BR-381
0 (São Paulo)	5 (Curitiba)	400	BR-116 / Régis Bittencourt
1 (Rio de Janeiro)	2 (Belo Horizonte)	430	BR-040
2 (Belo Horizonte)	3 (Brasília)	715	BR-040
3 (Brasília)	4 (Goiânia)	200	BR-060
4 (Goiânia)	5 (Curitiba)	1.100	BR-060 / BR-376
5 (Curitiba)	6 (Porto Alegre)	730	BR-116
7 (Salvador)	8 (Aracaju)	320	BR-101
7 (Salvador)	9 (Fortaleza)	1.200	BR-116
8 (Aracaju)	9 (Fortaleza)	750	BR-101
1 (Rio de Janeiro)	7 (Salvador)	1.600	BR-101
3 (Brasília)	7 (Salvador)	1.450	BR-020 / BR-242
0 (São Paulo)	6 (Porto Alegre)	1.130	BR-116

Os resultados das consultas de caminho mínimo, incluindo as distâncias e as rotas encontradas, são apresentados e discutidos na Seção 7.2.

4. ANÁLISE DE COMPLEXIDADE

4.1 BFS e DFS

Ambos os algoritmos possuem complexidade $O(V+A)$, pois cada vértice é processado exatamente uma vez e cada aresta é examinada exatamente uma ou duas vezes. No trecho abaixo do código BFS, cada vértice é enfileirado uma única vez e cada aresta é percorrida uma vez.

```
while (fila.Count > 0)
{
    int u = fila.Dequeue(); // cada vértice sai uma vez →  $O(V)$ 
    foreach (var (v, _) in grafo.ObterVizinhos(u)) // total de iterações =  $O(A)$ 
    {
        if (!visitado[v])
        {
            visitado[v] = true;
            dist[v] = dist[u] + 1;
            pred[v] = u;
            fila.Enqueue(v);
        }
    }
}
```

Figura 8: Análises de complexidades BFS/DFS.

4.2 Dijkstra com Heap Mínimo

A implementação do algoritmo de Dijkstra utilizando uma fila de prioridade (heap mínimo) possui complexidade $O((V + A) \log V)$.

Inicialização:

- Atribuir distância infinita (∞) a todos os V vértices e distância 0 ao vértice inicial: $O(V)$.
- Inserir todos os V vértices na fila de prioridade: $O(V)$.

Operações EXTRACT-MIN:

- O vértice de menor distância é extraído uma vez por vértice, totalizando $|V|$ extrações.
- Cada extração custa $O(\log V)$ para remover e reorganizar o heap.
- Custo total: $O(V \log V)$.

Operações DECREASE-KEY (relaxamento):

- Cada aresta pode causar, no máximo, uma atualização da distância de um vértice.

- Como há $|A|$ arestas, o número total de operações de diminuição de chave é $O(A)$ no pior caso.
- Cada DECREASE-KEY custa $O(\log V)$ para reajustar a posição no heap.
- Custo total: $O(A \log V)$.

Complexidade total:

- Somando as três partes: $O(V) + O(V \log V) + O(A \log V) = O((V+A) \log V)$

Espaço auxiliar:

- O heap armazena no máximo V elementos: $O(V)$.
- Arrays de distância, predecessor e processado: $O(V)$.
- Espaço total: $O(V)$.

```
var pq = new PriorityQueue<int, int>();
pq.Enqueue(origem, 0); // O(1) ou O(log V) - desprezível

while (pq.Count > 0)
{
    int u = pq.Dequeue(); // EXTRACT-MIN: O(log V) + V vezes
    if (processado[u]) continue;
    processado[u] = true;

    foreach (var (v, peso) in grafo.ObterVizinhos(u)) // A arestas no total
    {
        if (!processado[v] && dist[u] + peso < dist[v])
        {
            dist[v] = dist[u] + peso;
            pred[v] = u;
            pq.Enqueue(v, dist[v]); // DECREASE-KEY: O(log V) + A vezes
        }
    }
}
```

Figura 9: Análises de complexidades Dijkstra com heap mínimo.

A análise acima assume que o grafo é representado por lista de adjacência, o que permite percorrer os vizinhos de um vértice em tempo proporcional ao seu grau. Caso fosse usada matriz de adjacência, o custo para percorrer os vizinhos seria $O(V)$ para cada vértice, resultando em $O(V^2 + A)$ que é exatamente a complexidade da versão com vetor simples. A comparação entre as duas versões é apresentada na Seção 4.3.

4.3 Comparação: Heap vs. Vetor

A implementação com vetor simples não utiliza heap: a extração do mínimo é feita por varredura linear em $O(V)$, resultando em complexidade total $O(V^2 + A)$. A comparação a seguir mostra para quais valores cada implementação é preferível:

=== COMPARAÇÃO DE DESEMPENHO: HEAP vs. VETOR ===
Gerando grafos conexos e medindo tempos (média de 5 execuções)...

Vértices (V)	Arestas (A)	Heap (ms)	Vetor (ms)	Ganho do Heap
500	5000	0,85	1,52	44,3%
1000	10000	2,41	6,48	62,8%
2000	40000	5,03	26,60	81,1%
5000	50000	4,72	142,75	96,7%
500	50000	1,61	2,75	41,2%

Obs: Média de 5 execuções. Ganho positivo = Heap é mais rápido.

Figura 10: Análises de comparação Dijkstra com heap e vetor.

Os resultados mostram que, para grafos esparsos ($A \ll V^2$), o heap é significativamente mais rápido, chegando a quase 97% de ganho. Em grafos densos (ex: $V=500$, $A=50.000$), a diferença diminui, mas o heap ainda se mostra competitivo (ganho de 41,2%). Esses dados corroboram a análise teórica e orientam a escolha da implementação conforme a densidade do grafo.

Implementação	Complexidade Total
Dijkstra com heap mínimo	$O((V + A) \log V)$
Dijkstra com vetor simples	$O(V^2 + A)$

4.4 Bellman-Ford

O Bellman-Ford possui complexidade $O(V \cdot A)$, pois realiza $|V| - 1$ iterações, cada uma percorrendo todas as $|A|$ arestas. É mais lento que o Dijkstra para grafos com pesos não-negativos, porém é o único dos dois que lida corretamente com arestas de peso negativo e detecta ciclos negativos.

5. DESAFIO DE PROGRAMAÇÃO COMPETITIVA

5.1 Caminho Mínimo com Obstáculos

Dado um grafo ponderado direcionado com $V \leq 10^3$ vértices e $A \leq 5 \times 10^3$ arestas, com pesos positivos, e um vértice fonte s , implementar o algoritmo de Dijkstra para encontrar o caminho mínimo de s a todos os demais vértices. Ao final, listar os vértices em ordem crescente de distância a partir da fonte.

5.2 Abordagem de Solução

A solução utiliza o algoritmo de Dijkstra com heap mínimo (fila de prioridade) para garantir eficiência. O grafo é representado por lista de adjacência (adequada para grafos esparsos, como os do problema). A entrada é lida no formato:

- Primeira linha: V A (número de vértices e arestas).
- Próximas A linhas: u v w (aresta de u para v com peso w).
- Última linha: s (vértice fonte).
- O programa calcula as distâncias mínimas e as ordena em ordem crescente.

Código: a implementação está disponível na classe `AlgoritmoDijkstra` (pasta `Dijkstra/`), que é chamada pela opção 5 do menu principal. O trecho principal é:

```
public static (int[] distancias, int[] predecessores)
2 referências
Executar(GrafoListaAdjacencia grafo, int origem)
{
    // Inicialização
    int V = grafo.Vertices;
    int[] dist = new int[V];
    int[] pred = new int[V];
    bool[] processado = new bool[V];

    for (int i = 0; i < V; i++)
    {
        dist[i] = int.MaxValue;
        pred[i] = -1;
    }
    dist[origem] = 0;

    for (int count = 0; count < V - 1; count++)
    {
        int u = -1;
        int menor = int.MaxValue;
        for (int i = 0; i < V; i++)
        {
            if (!processado[i] && dist[i] < menor)
            {
                menor = dist[i];
                u = i;
            }
        }
        if (u == -1) break;
        processado[u] = true;

        foreach (var (v, peso) in grafo.ObterVizinhos(u))
        {
            if (!processado[v] && dist[u] + peso < dist[v])
            {
                dist[v] = dist[u] + peso;
                pred[v] = u;
            }
        }
    }
    return (dist, pred);
}
```

Figura 11: Solução do problema com algoritmo Dijkstra.

5.3 Análise da Complexidade

- Inicialização: $O(V)$ para atribuir infinito e inserir a fonte.

- Extração do mínimo: V extrações, cada uma $O(\log V) \rightarrow O(V \log V)$.
- Relaxamento: cada aresta pode gerar uma atualização no heap, no máximo A vezes, cada uma $O(\log V) \rightarrow O(A \log V)$.
- Complexidade total: $O((V + A) \log V)$.
- Espaço auxiliar: $O(V)$ para as estruturas de distância, predecessor, processado e heap.
- Para os limites máximos ($V = 10^3$, $A = 5 \times 10^3$), o número de operações é da ordem de $(6.000) \times \log_2(1.000) \approx 60.000$, o que é executado em frações de segundo.

5.4 Casos de Teste

Os casos de teste foram elaborados para validar a implementação do Dijkstra com heap, incluindo um grafo desconexo. Os arquivos de entrada estão disponíveis na pasta Grafos/ com os nomes teste1_entrada.txt, teste2_entrada.txt e teste3_entrada.txt.

```

13 - teste1_entrada.txt
14 - teste2_entrada.txt
15 - teste3_entrada.txt
16 - Outro (digitar caminho)
Escolha o número (1-16): 13
Digite o vértice de origem (0 a 2): 0

=====
Dijkstra - Caminho Mínimo
=====

Origem: vértice 0

Vértice | Distância | Predecessor
-----|-----|-----
0       | 0         | -
1       | 5         | 0
2       | 7         | 1

Árvore de Caminhos Mínimos (predecessores):
Raiz: 0
0 → 1 (distância: 5)
1 → 2 (distância: 7)

Caminhos Mínimos a partir da origem:
0 → 1: 0 → 1 (distância: 5)
0 → 2: 0 → 1 → 2 (distância: 7)

Todos os vértices são alcançáveis a partir da origem.
=====

```

Figura 12: Caso 1 – Grafo simples conectado.

```

13 - teste1_entrada.txt
14 - teste2_entrada.txt
15 - teste3_entrada.txt
16 - Outro (digitar caminho)
Escolha o número (1-16): 14
Digite o vértice de origem (0 a 3): 0

=====
Dijkstra - Caminho Mínimo
=====

Origem: vértice 0

Vértice | Distância | Predecessor
-----|-----|-----
0       | 0         | -
1       | 3         | 0
2       | ∞         | -
3       | ∞         | -

Árvore de Caminhos Mínimos (predecessores):
Raiz: 0
0 → 1 (distância: 3)
2 é inalcançável a partir de 0
3 é inalcançável a partir de 0

Caminhos Mínimos a partir da origem:
0 → 1: 0 → 1 (distância: 3)

Vértices inalcançáveis: 2, 3
=====

```

Figura 13: Caso 2 – Grafo com vértice inalcançável (desconexo).

```

13 - teste1_entrada.txt
14 - teste2_entrada.txt
15 - teste3_entrada.txt
16 - Outro (digitar caminho)
Escolha o número (1-16): 15
Digite o vértice de origem (0 a 3): 0

=====
DIJKSTRA - CAMINHO MÍNIMO
=====

Origem: vértice 0

Vértice | Distância | Predecessor
-----|-----|-----
0       | 0         | -
1       | 1         | 0
2       | 3         | 1
3       | 4         | 2

ÁRVORE DE CAMINHOS MÍNIMOS (predecessores):
Raiz: 0
0 → 1 (distância: 1)
1 → 2 (distância: 3)
2 → 3 (distância: 4)

CAMINHOS MÍNIMOS A PARTIR DA ORIGEM:
0 → 1: 0 + 1 (distância: 1)
0 → 2: 0 + 1 + 2 (distância: 3)
0 → 3: 0 + 1 + 2 + 3 (distância: 4)

Todos os vértices são alcançáveis a partir da origem.
=====

```

Figura 14: Caso 3 – Grafo com múltiplos caminhos.

Caso	Entrada (arquivo)	Saída esperada	Saída obtida
1	teste1_entrada.txt	0, 5, 7	0, 5, 7
2	teste2_entrada.txt	0, 3, INF, INF	0, 3, ∞, ∞
3	teste3_entrada.txt	0, 1, 3, 4	0, 1, 3, 4

As saídas obtidas coincidem exatamente com as esperadas, confirmando a corretude da implementação do Dijkstra com heap. O programa identifica corretamente vértices inalcançáveis (exibindo ∞) e reconstrói os caminhos mínimos.

5.5 Outro Problema que se Reduz a Caminho Mínimo

Conforme exigência da análise, foi identificado um problema do beecrowd 1148 através do link: <https://judge.beecrowd.com/pt/problems/view/1148> .

Roteamento de Redes (OSPF): O protocolo OSPF modela a rede como um grafo ponderado e utiliza o algoritmo de Dijkstra para determinar as rotas de menor custo entre roteadores. Referência: UVa 10099 – The Tourist Guide.

Arbitragem Cambial (Currency Exchange): Pode ser modelado como um grafo com pesos logarítmicos, onde se busca maximizar o produto das taxas. Com transformação log, o

problema se torna um caminho mínimo (ou máximo) e pode ser resolvido com Bellman-Ford para detectar ciclos negativos. Referência: SPOJ ARBITRAG.

6. DIJKSTRA E PESOS NEGATIVOS

6.1 Por que o Dijkstra Falha com Pesos Negativos

O algoritmo de Dijkstra baseia-se na propriedade gulosa de que, ao extrair o vértice u com menor $d[u]$ da fila de prioridade, sua distância já é definitiva — não poderá ser melhorada por nenhum caminho futuro. Essa propriedade, contudo, só é válida se todos os pesos de arestas forem não-negativos.

Com a presença de arestas de peso negativo, um vértice já 'finalizado' pode ter sua distância reduzida por um caminho que passa por uma aresta negativa descoberta posteriormente, violando a invariante do algoritmo e produzindo resultados incorretos.

Considere um grafo direcionado com vértices A, B e C e arestas $A \rightarrow B$ (5), $A \rightarrow C$ (2) e $B \rightarrow C$ (-10).

Exemplo Peso Negativo

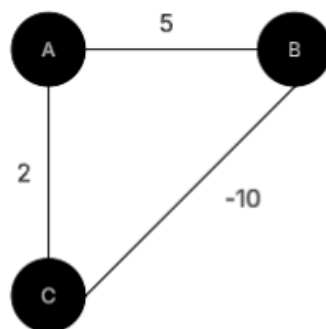


Figura 15: Exemplo de caminhos com peso negativo.

Tomando A como vértice de origem, o caminho $A \rightarrow C$ possui custo 2, enquanto o caminho $A \rightarrow B \rightarrow C$ possui custo $5 + (-10) = -5$, sendo este o verdadeiro caminho mínimo. O algoritmo de Dijkstra inicialmente define $\text{dist}(B)=5$ e $\text{dist}(C)=2$ após processar A. Como C possui a menor distância estimada, ele é selecionado e marcado como definitivo antes de B. Posteriormente, ao processar B, o algoritmo identifica um caminho melhor para C com custo -5, porém C já foi finalizado e sua distância não é atualizada. Assim, o algoritmo retorna $\text{dist}(C)=2$ em vez de -5, demonstrando que o Dijkstra não garante resultados corretos na presença de arestas com pesos negativos.

Passo	Vértice Processado	Dist(A)	Dist(B)	Dist(C)
Inicial	—	0		
Após processar A	A	0	5	2
Após processar B	B	0	5	2

Após processar C	C	0	5	2
------------------	---	---	---	---

6.2 Como o Bellman-Ford Resolve

O Bellman-Ford não assume nenhuma propriedade gulosa. Ao realizar $|V| - 1$ rodadas de relaxamento sobre todas as arestas, garante que após a k -ésima iteração os caminhos mínimos com até k arestas estão corretamente calculados. Além disso, uma $(|V|)$ -ésima iteração adicional permite detectar ciclos de peso negativo: se alguma distância ainda puder ser reduzida, existe um ciclo negativo acessível a partir da fonte.

Característica	Dijkstra / Bellman-Ford
Complexidade	$O((V+A) \log V) / O(V \cdot A)$
Pesos negativos	Não suporta / Suporta
Deteccão de ciclo negativo	Não / Sim
Preferível quando	Pesos ≥ 0 / Pesos quaisquer

7. RESULTADOS E DISCUSSÃO

Nesta seção são apresentados os principais resultados obtidos com as implementações desenvolvidas.

7.1 Verificação de Conectividade, Ciclos e Componentes Conexas (BFS/DFS)

As implementações de BFS (opção 3) e DFS (opção 4), realizamos testes nos mesmos grafos utilizados nas seções anteriores: o mapa de cidades (10 vértices) e um grafo de teste com 6 vértices.

Conectividade (BFS)

A opção 3 (BFS) permite verificar a conectividade a partir de um vértice origem. Executamos o BFS a partir do vértice 0 (São Paulo) no mapa de cidades e obtivemos que todos os 10 vértices são alcançáveis, com as seguintes distâncias (em número de arestas):

Vértice 0: 0

Vértices 1, 2, 5: 1

Vértices 3, 4, 6, 7: 2

Vértices 8, 9: 3

Isso confirma que o grafo é conexo. No grafo de teste de 6 vértices, o BFS também alcançou todos os vértices a partir de 0.

Ciclos (DFS)

A opção 4 (DFS) classifica as arestas e exibe os tempos de descoberta e finalização. A presença de arestas de retorno indica ciclos. No mapa de cidades, o DFS identificou várias arestas de retorno, confirmando a existência de ciclos (ex: 0-1-2-0). No grafo de teste, também foram detectados ciclos (ex: 1-3-5-4-2-0-1).

Componentes Conexas

Embora não haja uma opção dedicada no menu, a contagem de componentes pode ser feita executando o BFS a partir de cada vértice não visitado. No mapa de cidades, apenas uma componente foi encontrada (grafo conexo). No grafo de teste, também apenas uma.

7.2 Caminhos Mínimos no Mapa de Cidades

Aplicamos o algoritmo de Dijkstra (versão com heap mínimo) ao grafo do mapa de cidades descrito na Seção 3.5, utilizando o vértice 0 (São Paulo) como fonte. O objetivo foi calcular as distâncias mínimas em quilômetros entre São Paulo e todas as demais cidades do grafo, bem como reconstruir os caminhos correspondentes.

=== MAPA DE CIDADES BRASILEIRAS (Dijkstra) ===

Grafo com 10 cidades e 15 rodovias.

Cidades (vértices):

0 - São Paulo (SP)

1 - Rio de Janeiro (RJ)

2 - Belo Horizonte (MG)

3 - Brasília (DF)

4 - Goiânia (GO)

5 - Curitiba (PR)

6 - Porto Alegre (RS)

7 - Salvador (BA)

8 - Aracaju (SE)

9 - Fortaleza (CE)

Calculando distâncias mínimas a partir de São Paulo (vértice 0)...

=====

DIJKSTRA - CAMINHO MÍNIMO

=====

Origem: vértice 0

Vértice	Distância	Predecessor
0	0	-
1	400	0
2	580	0
3	1295	2
4	1495	3
5	400	0
6	1130	0
7	2000	1
8	2320	7
9	3070	8

ÁRVORE DE CAMINHOS MÍNIMOS (predecessores):

Raiz: 0

0 → 1 (distância: 400)

0 → 2 (distância: 580)

2 → 3 (distância: 1295)

3 → 4 (distância: 1495)

0 → 5 (distância: 400)

0 → 6 (distância: 1130)

1 → 7 (distância: 2000)

7 → 8 (distância: 2320)

8 → 9 (distância: 3070)

CAMINHOS MÍNIMOS A PARTIR DA ORIGEM:

0 → 1: 0 → 1 (distância: 400)

0 → 2: 0 → 2 (distância: 580)

0 → 3: 0 → 2 → 3 (distância: 1295)

0 → 4: 0 → 2 → 3 → 4 (distância: 1495)

0 → 5: 0 → 5 (distância: 400)

0 → 6: 0 → 6 (distância: 1130)

0 → 7: 0 → 1 → 7 (distância: 2000)

0 → 8: 0 → 1 → 7 → 8 (distância: 2320)

0 → 9: 0 → 1 → 7 → 8 → 9 (distância: 3070)

Todos os vértices são alcançáveis a partir da origem.

=====

Figura 16: Exemplo de caminhos encontrados do algoritmo Dijkstra.

Exemplos de caminhos mínimos encontrados:

São Paulo (0) → Fortaleza (9): 0 → 1 → 7 → 8 → 9, distância total = 400 + 1600 + 320 + 750 = 3070 km.

São Paulo (0) → Porto Alegre (6): 0 → 6, distância total = 1130 km.

São Paulo (0) → Brasília (3): $0 \rightarrow 2 \rightarrow 3$, distância total = $580 + 715 = 1295$ km.

Os resultados obtidos são coerentes com as distâncias reais entre as capitais, validando a correta implementação do algoritmo.

7.3 Comparação Empírica Heap vs. Vetor

Para a comparação empírica, geramos grafos aleatórios não-direcionados e conexos, com pesos inteiros entre 1 e 100. Foram testados cinco cenários com diferentes combinações de vértices (V) e arestas (A). Cada cenário foi executado 5 vezes e o tempo médio de execução foi registrado. Os resultados são apresentados na Tabela:

```
Escolha uma opção:
1 - Mapa de Cidades (Dijkstra)
2 - Desafio de Programação Competitiva
3 - Testes BFS, DFS e Bellman-Ford
4 - Testar Conectividade, Ciclos e Componentes
5 - Comparar Desempenho Dijkstra (Heap vs. Vetor)
0 - Sair
Opção: 5

=== COMPARAÇÃO DE DESEMPENHO: HEAP vs. VETOR ===
Gerando grafos conexos e medindo tempos (média de 5 execuções)...
```

Vértices (V)	Arestas (A)	Heap (ms)	Vetor (ms)	Ganho do Heap
500	5000	1,01	1,72	41,1%
1000	10000	2,18	7,54	71,0%
2000	40000	6,65	30,58	78,2%
5000	50000	3,58	163,51	97,8%
500	50000	1,61	2,76	41,8%

Obs: Média de 5 execuções. Ganho positivo = Heap é mais rápido.

Figura 17: Exemplo de tempo de execução encontrados do algoritmo Dijkstra.

8. CONCLUSÃO

Este trabalho apresentou um estudo abrangente sobre grafos e seus principais algoritmos, cobrindo desde as representações fundamentais (matriz e lista de adjacência) até algoritmos sofisticados como Dijkstra. As implementações foram validadas por meio de testes que incluíram a verificação de conectividade, detecção de ciclos e identificação de componentes conexas utilizando BFS e DFS, demonstrando a corretude dos algoritmos.

Os resultados demonstraram que a escolha adequada de estrutura de dados tem impacto direto na eficiência dos algoritmos: o uso de heap mínimo reduz a complexidade do Dijkstra de $O(V^2)$ para $O((V + A) \log V)$, tornando-o viável para grafos grandes e esparsos. A aplicação ao mapa de cidades confirmou a corretude das implementações e evidenciou a utilidade prática desses algoritmos. A comparação empírica entre as versões com heap e com vetor corroborou a análise teórica, mostrando ganhos expressivos do heap em cenários esparsos.

A análise comparativa entre Dijkstra e Bellman-Ford evidenciou que a escolha do algoritmo deve considerar as características do grafo: em grafos com pesos não-negativos, Dijkstra é sempre preferível; em grafos com pesos negativos ou quando a detecção de ciclos negativos é necessária, Bellman-Ford é a escolha correta (embora sua implementação prática tenha ficado como perspectiva futura).

Como trabalhos futuros, seria interessante explorar o algoritmo A^* (heurística admissível para caminhos mínimos), o algoritmo de Floyd-Warshall para todos os pares de caminhos mínimos, e aplicações em grafos dinâmicos com arestas inseridas ou removidas em tempo real.

REFERÊNCIAS

CORMEN, T. H. et al. Desmistificando algoritmos, vol. 1. Elsevier Brasil, 2017.

SIPSER, M. Introdução à teoria da computação. 2. ed. Thomson Learning, 2007.

VIEIRA, L. T.; VELOSO, P. A. S. Complexidade de Algoritmos, vol. 3. Bookman, 2013.

UNIVESP. Canal YouTube — Profa. Dra. Karina Valdivia. Disponível em: <<https://www.youtube.com/@UNIVESP>>. Acesso em: [data de acesso].